Search...

Maximum Subarray Sum in a given Range

Last Updated : 14 Mar, 2023

Given an array of n numbers, the task is to answer the following queries:

```
maximumSubarraySum(start, end) : Find the maximum
subarray sum in the range from array index 'start'
to 'end'.
```

Also see : Range Query With Update Required

Examples:

```
Input : arr[] = {1, 3, -4, 5, -2}
        Query 1: start = 0, end = 4
        Query 2: start = 0, end = 2
Output : 5
         4
Explanation:
For Query 1, [1, 3, -4, 5] or ( [5] )
represent the maximum sum sub arrays
with sum = 5.

For Query 2, [1, 3] represents the
maximum sum subarray in the query range
with sum = 4
```

Segment Trees can be used to solve this problem. Here, we need to keep information regarding various cumulative sums. At every Node we store the following:

1. Maximum Prefix Sum,
2. Maximum Suffix Sum,
3. Total Sum

We use cookies to ensure you have the best browsing experience on our website. By using our site, you acknowledge that you have read and understood our Cookie Policy & Privacy Policy

Got It !

A classical Segment Tree with each Node storing the above information should be enough to answer each query. The only focus here is on how the left and the right Nodes of the tree are merged together. Now, we will discuss how each of the information is constructed in each of the segment tree Nodes using the information of its left and right child.

Constructing the Maximum Prefix Sum using Left and Right child

There can be two cases for maximum prefix sum of a Node:

- The maximum prefix sum occurs in the left child,

Case 1: Maximum Prefix Sum = 3

```
In this Case,
Maximum Prefix Sum = Maximum Prefix Sum of Left Child
```

- The maximum prefix sum contains every array element of the left child and the elements contributing to the maximum prefix sum of the right child,

Case 2: Maximum Prefix Sum = 3 + (-4) + 2 + 1 + 4 = 6

```
In this Case,
Maximum Prefix Sum = Total Sum of Left Child +
                     Maximum Prefix Sum of Right Child
```

Constructing the Maximum Suffix Sum using Left and Right child

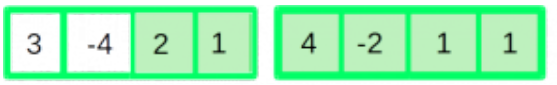
There can be two cases for maximum suffix sum of a Node:

- The maximum suffix sum occurs in the right child,

We use cookies to ensure you have the best browsing experience on our website. By using our site, you acknowledge that you have read and understood our Cookie Policy & Privacy Policy

```
In this Case,
Maximum Suffix Sum = Maximum Suffix Sum of Right Child
```

- The maximum suffix sum contains every array element of the Right child and the elements contributing to the maximum suffix sum of the left child,



Case 2: Maximum Suffix Sum = 2 + 1 + 4 + (-2) + 1 + 1 = 7

```
In this Case,
Maximum Suffix Sum = Total Sum of Right Child +
                     Maximum Suffix Sum of Left Child
```

Constructing the Maximum Subarray Sum using Left and Right child

There can be three cases for the maximum sub-array sum of a Node:

- The maximum sub-array sum occurs in the left child,

Case 2: Maximum Subarray Sum = 3

```
In this Case,
Maximum Sub-array Sum = Maximum Subarray Sum of Left Child
```

- The maximum sub-array sum occurs in the right child,

We use cookies to ensure you have the best browsing experience on our website. By using our site, you acknowledge that you have read and understood our Cookie Policy & Privacy Policy

```
In this Case,
Maximum Sub-array Sum = Maximum Subarray Sum of Right Child
```

- The maximum subarray sum, contains array elements of the right child contributing to the maximum prefix sum of the right child, and the array elements of the Left child contributing to the maximum suffix sum of the left child,

Case 3: Maximum Subarray Sum = 2 + 1 + 4
= 7

```
In this Case,
Maximum Subarray Sum = Maximum Prefix Sum of Right Child
                                    +
                       Maximum Suffix Sum of Left Child
```

Implementation:

CPP Java Python3 C# JavaScript

```cpp
// C++ Program to Implement Maximum Sub-Array Sum in a range
#include <bits/stdc++.h>
using namespace std;

#define inf 0x3f3f

/* Node of the segment tree consisting of:
1. Maximum Prefix Sum,
2. Maximum Suffix Sum,
3. Total Sum,
4. Maximum Sub-Array Sum */
struct Node {
    int maxPrefixSum;
    int maxSuffixSum;
```

We use cookies to ensure you have the best browsing experience on our website. By using our site, you acknowledge that you have read and understood our Cookie Policy & Privacy Policy

```
    Node()
    {
        maxPrefixSum = maxSuffixSum = maxSubarraySum = -inf;
        totalSum = -inf;
```



```
};

// Returns Parent Node after merging its left and right child
Node merge(Node leftChild, Node rightChild)
{
    Node parentNode;
    parentNode.maxPrefixSum = max(leftChild.maxPrefixSum,
                                  leftChild.totalSum +
                                  rightChild.maxPrefixSum);

    parentNode.maxSuffixSum = max(rightChild.maxSuffixSum,
                                  rightChild.totalSum +
                                  leftChild.maxSuffixSum);

    parentNode.totalSum = leftChild.totalSum +
                          rightChild.totalSum;

    parentNode.maxSubarraySum = max({leftChild.maxSubarraySum,
                                     rightChild.maxSubarraySum,
                                     leftChild.maxSuffixSum +
                                     rightChild.maxPrefixSum});

    return parentNode;
}

// Builds the Segment tree recursively
void constructTreeUtil(Node* tree, int arr[], int start,
                                    int end, int index)
{

    /* Leaf Node */
    if (start == end) {

        // single element is covered under this range
        tree[index].totalSum = arr[start];
```

We use cookies to ensure you have the best browsing experience on our website. By using our site, you acknowledge that you have read and understood our Cookie Policy & Privacy Policy

```
        tree[index].maxSubarraySum = arr[start];
        return;
    }

    // Recursively Build left and right children
    int mid = (start + end) / 2;
    constructTreeUtil(tree, arr, start, mid, 2 * index);
    constructTreeUtil(tree, arr, mid + 1, end, 2 * index + 1);

    // Merge left and right child into the Parent Node
    tree[index] = merge(tree[2 * index], tree[2 * index + 1]);
}

/* Function to construct segment tree from given array.
   This function allocates memory for segment tree and
   calls constructTreeUtil() to fill the allocated
   memory */
Node* constructTree(int arr[], int n)
{
    // Allocate memory for segment tree
    int x = (int)(ceil(log2(n))); // Height of the tree

    // Maximum size of segment tree
    int max_size = 2 * (int)pow(2, x) - 1;
    Node* tree = new Node[max_size];

    // Fill the allocated memory tree
    constructTreeUtil(tree, arr, 0, n - 1, 1);

    // Return the constructed segment tree
    return tree;
}

/* A Recursive function to get the desired
   Maximum Sum Sub-Array,
The following are parameters of the function-

tree     --> Pointer to segment tree
index --> Index of the segment tree Node
ss & se  --> Starting and ending indexes of the
             segment represented by
```

We use cookies to ensure you have the best browsing experience on our website. By using our site, you acknowledge that you have read and understood our Cookie Policy & Privacy Policy

```
Node queryUtil(Node* tree, int ss, int se, int qs,
                                int qe, int index)
{
    // No overlap
    if (ss > qe || se < qs) {

        // returns a Node for out of bounds condition
        Node nullNode;
        return nullNode;
    }

    // Complete overlap
    if (ss >= qs && se <= qe) {
        return tree[index];
    }

    // Partial Overlap Merge results of Left
    // and Right subtrees
    int mid = (ss + se) / 2;
    Node left = queryUtil(tree, ss, mid, qs, qe,
                                    2 * index);
    Node right = queryUtil(tree, mid + 1, se, qs,
                            qe, 2 * index + 1);

    // merge left and right subtree query results
    Node res = merge(left, right);
    return res;
}

/* Returns the Maximum Subarray Sum between start and end
   It mainly uses queryUtil(). */
int query(Node* tree, int start, int end, int n)
{
    Node res = queryUtil(tree, 0, n - 1, start, end, 1);
    return res.maxSubarraySum;
}

int main()
{
    int arr[] = { 1, 3, -4, 5, -2 };
    int n = sizeof(arr) / sizeof(arr[0]);
```

We use cookies to ensure you have the best browsing experience on our website. By using our site, you acknowledge that you have read and understood our Cookie Policy & Privacy Policy

```
    Node* Tree = constructTree(arr, n);
    int start, end, maxSubarraySum;

    // Answering query 1:
    start = 0;
    end = 4;
    maxSubarraySum = query(Tree, start, end, n);
    cout << "Maximum Sub-Array Sum between "
         << start << " and " << end
         << " = " << maxSubarraySum << "\n";

    // Answering query 2:
    start = 0;
    end = 2;
    maxSubarraySum = query(Tree, start, end, n);
    cout << "Maximum Sub-Array Sum between "
         << start << " and " << end
         << " = " << maxSubarraySum << "\n";

    return 0;
}
```

Output:

```
Maximum Sub-Array Sum between 0 and 4 = 5
Maximum Sub-Array Sum between 0 and 2 = 4
```

Time Complexity: O(logn) for each query.

Comment More info Advertise with us

Next Article

Maximum Subarray Sum in a given Range

Similar Reads

We use cookies to ensure you have the best browsing experience on our website. By using our site, you acknowledge that you have read and understood our Cookie Policy & Privacy Policy

DSA (Data Structures and Algorithms) is the study of organizing data efficiently using data structures like arrays, stacks, and trees, paired with step-by-step procedures (or algorithms) to solve problems effectively....

7 min read

Quick Sort

QuickSort is a sorting algorithm based on the Divide and Conquer that picks an element as a pivot and partitions the given array around the picked pivot by placing the pivot in its correct position in the sorted arra...

12 min read

Merge Sort - Data Structure and Algorithms Tutorials

Merge sort is a popular sorting algorithm known for its efficiency and stability. It follows the divide-and-conquer approach. It works by recursively dividing the input array into two halves, recursively sorting the tw...

14 min read

SQL Commands | DDL, DQL, DML, DCL and TCL Commands

SQL commands are crucial for managing databases effectively. These commands are divided into categories such as Data Definition Language (DDL), Data Manipulation Language (DML), Data Control Language (DCL),...

7 min read

Bubble Sort Algorithm

Bubble Sort is the simplest sorting algorithm that works by repeatedly swapping the adjacent elements if they are in the wrong order. This algorithm is not suitable for large data sets as its average and worst-case...

8 min read

Data Structures Tutorial

Data structures are the fundamental building blocks of computer programming. They define how data is organized, stored, and manipulated within a program. Understanding data structures is very important for...

2 min read

Breadth First Search or BFS for a Graph

Given a undirected graph represented by an adjacency list adj, where each adj[i] represents the list of vertices connected to vertex i. Perform a Breadth First Search (BFS) traversal starting from vertex 0, visiting vertices...

15+ min read

Binary Search Algorithm - Iterative and Recursive Implementation

Binary Search Algorithm is a searching algorithm used in a sorted array by repeatedly dividing the search interval in half. The idea of binary search is to use the information that the array is sorted and reduce the tim...

15 min read

Insertion Sort Algorithm

Insertion sort is a simple sorting algorithm that works by iteratively inserting each element of an unsorted list

We use cookies to ensure you have the best browsing experience on our website. By using our site, you acknowledge that you have read and understood our Cookie Policy & Privacy Policy

Dijkstra's Algorithm to find Shortest Paths from a Source to all

Given a weighted undirected graph represented as an edge list and a source vertex src, find the shortest path distances from the source vertex to all other vertices in the graph. The graph contains V vertices, numbered...

12 min read

GeeksforGeeks
Sanchhaya Education Private Limited

Corporate & Communications Address:
A-143, 7th Floor, Sovereign Corporate Tower, Sector- 136, Noida, Uttar Pradesh (201305)

Registered Address:
K 061, Tower K, Gulshan Vivante Apartment, Sector 137, Noida, Gautam Buddh Nagar, Uttar Pradesh, 201305

GET IT ON Google Play
Download on the App Store

Advertise with us

Company
About Us
Legal
Privacy Policy
In Media
Contact Us
Advertise with us
GFG Corporate Solution
Placement Training Program

Languages
Python
Java
C++
PHP
GoLang
SQL
R Language
Android Tutorial
Tutorials Archive

DSA
Data Structures
Algorithms
DSA for Beginners
Basic DSA Problems

Data Science & ML
Data Science With Python
Data Science For Beginner
Machine Learning
ML Maths

We use cookies to ensure you have the best browsing experience on our website. By using our site, you acknowledge that you have read and understood our Cookie Policy & Privacy Policy

DSA Roadmap by Sandeep Jain
All Cheat Sheets

NumPy
NLP
Deep Learning

Web Technologies

HTML
CSS
JavaScript
TypeScript
ReactJS
NextJS
Bootstrap
Web Design

Python Tutorial

Python Programming Examples
Python Projects
Python Tkinter
Python Web Scraping
OpenCV Tutorial
Python Interview Question
Django

Computer Science

Operating Systems
Computer Network
Database Management System
Software Engineering
Digital Logic Design
Engineering Maths
Software Development
Software Testing

DevOps

Git
Linux
AWS
Docker
Kubernetes
Azure
GCP
DevOps Roadmap

System Design

High Level Design
Low Level Design
UML Diagrams
Interview Guide
Design Patterns
OOAD
System Design Bootcamp
Interview Questions

Inteview Preparation

Competitive Programming
Top DS or Algo for CP
Company-Wise Recruitment Process
Company-Wise Preparation
Aptitude Preparation
Puzzles

School Subjects

Mathematics
Physics
Chemistry
Biology
Social Science
English Grammar
Commerce
World GK

GeeksforGeeks Videos

DSA
Python
Java
C++
Web Development
Data Science
CS Subjects

@GeeksforGeeks, Sanchhaya Education Private Limited, All rights reserved

We use cookies to ensure you have the best browsing experience on our website. By using our site, you acknowledge that you have read and understood our Cookie Policy & Privacy Policy